

Ludwig-Maximilians-Universität München

Faculty of Mathematics, Informatics and Statistics

Department of Computer Science

Beyond Convergence: An Empirical Analysis of Learned Q-Functions in Model-Free RL

Jiacheng Chen, Juliette Garnier, Tommy Kiss,
Sifan Zhu

A Report Submitted in Partial
Fulfillment of the Requirements of

the Practical Course Applied Reinforcement Learning in Informatics

Group 6

Supervisors: Dr. Michel Tokic
Philipp Wissmann

September 13, 2026

Abstract

Model-free reinforcement learning algorithms use learned action-value functions to estimate future returns and optimise policies. Although these algorithms are usually evaluated based on policy performance, the accuracy and stability of their Q-functions are rarely examined directly. This study investigates whether learned Q-values reliably approximate realised returns, and whether their accuracy improves once policy performance has converged. Predicted Q-values were compared with discounted Monte Carlo returns from deterministic policy rollouts, with bias, temporal evolution and state dependence analysed across multiple random seeds and training checkpoints.

On the Pendulum-v1 task, all three algorithms eventually produce Q-values that are close to the empirical returns. The mean biases are 0.49, -3.65 and -7.21 for Deep Q-Network (DQN), Soft Actor-Critic (SAC) and Twin Delayed Deep Deterministic Policy Gradient (TD3), respectively. In contrast, DQN on CartPole-v1 exhibits substantial and persistent estimation errors despite near-optimal policy performance. Predicted values cluster around 65–70 for realised returns ranging from approximately 20 to 100, while relative bias remains at around -30% to -50% after policy performance has largely converged. Q-function stability also varies across the state space, with larger fluctuations in regions that are visited less frequently. These findings demonstrate that strong policy performance does not necessarily imply an accurate or converged Q-function. This highlights the importance of evaluating learned value functions alongside conventional policy-return metrics.

Contents

List of Figures	iii
1. Introduction	1
2. Related Work	2
3. Methodology	4
3.1. Training Setup and Hyperparameters	4
3.2. Ground-truth Q-value Estimation via Monte Carlo Rollouts	4
3.3. Error Metric	5
4. Experiments and Results	7
4.1. Convergence and Bias at End of Training	7
4.2. Policy Performance	9
4.3. Q-value against State across Environments	11
4.4. Evolution of Q-function	13
5. Discussion	15
5.1. Deviation from Theoretical Optimality	15
5.2. Error Localization in State Space	15
5.3. Q-Functions After Extended Training	16
5.4. Recurring Error and Instability Patterns	17
6. Conclusion and Outlook	18
6.1. Conclusion	18
6.2. Outlook	18
7. Bibliography	20
Appendix A. Additional Figures	21
Appendix B. Selected Algorithms	23
Appendix C. Benchmark Environments	25

List of Figures

4.1.	Training reward curves, averaged over 5 seeds (shaded: ± 1 std), together with the theoretical minimum and maximum achievable episode reward on each environment (Pendulum-v1: $-3254.7/0.0$; CartPole-v1: $0/500$).	7
4.2.	Estimated Q-value vs. Monte Carlo return for DQN on CartPole-v1, pooled over 5 seeds ($n = 50$ states per seed).	8
4.3.	Estimated Q-value vs. Monte Carlo return for DQN, SAC, and TD3 on Pendulum-v1, pooled over 5 seeds.	8
4.4.	Q-value bias over training on CartPole.	9
4.5.	Q-value bias over training on Pendulum.	10
4.6.	Re-evaluated mean reward with $n = 50$ over training across different seeds	10
4.7.	Training-time evaluation reward with $n = 5$ over a 5M-step run	11
4.8.	Q-value against a physically meaningful state variable (angular velocity for Pendulum, cart position for CartPole) for DQN over 1M steps, comparing best and final model checkpoints.	12
4.9.	Q-value against cart position of DQN on CartPole over 1M steps, at intermediate checkpoints.	13
4.10.	Evolution of learned Q-values for five fixed CartPole states during 5M steps of DQN training. Cart position and velocity are fixed to zero; pole angle and angular velocity vary across the five states shown in the panels. Both discrete actions are evaluated.	13
A.1.	Evolution of learned Q-values for five fixed Pendulum states during 1M steps of DQN training. States are represented by $(\theta, \dot{\theta})$; the corresponding three-dimensional network observation is $[\cos \theta, \sin \theta, \dot{\theta}]$	21
A.2.	Evolution of learned Q-values for five fixed Pendulum states during 1M steps of SAC training. States are represented by $(\theta, \dot{\theta})$; the corresponding network observation is $[\cos \theta, \sin \theta, \dot{\theta}]$	21
A.3.	Evolution of learned Q-values for five fixed Pendulum states during 1M steps of TD3 training. States are represented by $(\theta, \dot{\theta})$; the corresponding network observation is $[\cos \theta, \sin \theta, \dot{\theta}]$	21
A.4.	Relative Q-value bias (left) and re-evaluated mean reward with $n = 50$ (right) over training for DQN on Pendulum-v1, mean ± 1 std across 5 seeds.	22

List of Figures

- A.5. Relative Q-value bias (left) and re-evaluated mean reward with $n = 50$
(right) over training for SAC on Pendulum-v1, mean ± 1 std across 5 seeds. 22
- A.6. Relative Q-value bias (left) and re-evaluated mean reward with $n = 50$
(right) over training for TD3 on Pendulum-v1, mean ± 1 std across 5 seeds. 22

1. Introduction

Model-free reinforcement learning algorithms such as Deep Q-Learning (DQN), Soft Actor-Critic (SAC) and Twin Delayed Deep Deterministic Policy Gradient (TD3) learn a Q-function that estimates the expected return of an action, the policy is derived from these estimates by favoring actions with higher predicted return. Under tabular assumptions this process is guaranteed to converge to the optimal value function, but in practical training, policy performance can plateau, oscillate or collapse on simple benchmark tasks and the results are often very sensitive to random seeds and hyper-parameters. A common explanation for this gap is the "deadly triad" of function approximation, bootstrapping and off-policy learning, whose combination can cause value estimates to diverge even when the resulting policy still appears to perform reasonably (H. v. Hasselt, 2010).

Despite this, the Q-function itself is usually treated as a black box. Training progress is almost always judged by the return the policy achieves, while the value estimates are rarely inspected directly. It is therefore unclear whether a trained Q-function is a reliable estimate of the true value function, whether this reliability differs systematically between algorithms, and whether it continues to improve once policy performance has already converged.

This project investigates these questions empirically. Using a small set of representative model-free algorithms (DQN, SAC and TD3) and benchmark environments (CartPole and Pendulum), this project treats the algorithms as different instances of the same underlying mechanism, so that any observed error or instability pattern can be checked for whether it is specific to a single algorithm's design or shared across all of them. By comparing the Q-values predicted by the trained models against the Monte Carlo estimates of the return actually realized by their policies, this project characterizes what these learned Q-functions represent in practice, where in the state space they are least reliable and how they behave once policy performance has converged.

2. Related Work

Value-based methods scale beyond the tabular setting by replacing the lookup table with a function approximator. DQN (Mnih et al., 2015), which combines Q-learning with a deep neural network, scales to high-dimensional state spaces, while the action space is still discrete and small. A recurring issue with this line of work is that the max operator in the Q-learning update introduces a systematic positive bias whenever the underlying value estimates are noisy. Double Q-learning (H. v. Hasselt, 2010) first showed this in the tabular setting and proposed decoupling action selection from action evaluation using two independent estimators to remove the bias, and Double DQN (H. v. Hasselt et al., 2016) later extended the same idea to deep function approximation, demonstrating that the overestimation is not merely a theoretical concern but substantial in practice, occurring across a range of Atari games. Alongside the target network already used in DQN, this establishes decoupled estimation as one of the earliest and most widely reused stabilization techniques for Q-learning with function approximation. Building on this line of work, the deadly triad study (H. v. Hasselt et al., 2018) tests several Q-learning variants built on DQN’s experience-replay architecture and finds that unrealistic, transiently growing value estimates correlate with poor control performance. However, some runs with clearly unrealistic Q-values still perform well, since the relative ranking between action values remains intact. This is particularly relevant here, since it suggests that raw deviation between predicted and true Q-values may not directly translate into policy performance.

Actor-critic methods for continuous control face closely related problems. TD3 (Fujimoto et al., 2018) shows that the overestimation bias known from value-based methods also affects deterministic actor-critic algorithms and introduces three countermeasures: clipped double Q-learning, which takes the minimum over twin critics and directly extends the idea behind double q-learning to this setting; delayed policy updates, which update the actor less frequently than the critics to limit the accumulation of error; and target policy smoothing, which adds noise to the target action to prevent the critic from exploiting narrow, sharp peaks in its own value estimate. SAC (Haarnoja, Zhou, Abbeel, et al., 2018) approaches the same class of problems differently, casting policy learning as maximum-entropy reinforcement learning so that the actor maximizes expected return together with an entropy bonus, which the authors argue improves exploration and stabilizes off-policy training with a stochastic actor. A follow-up paper (Haarnoja, Zhou, Hartikainen, et al.,

2. Related Work

2018) simplifies the original formulation by removing its separate state-value network in favor of twin Q-critics with target networks, which is structurally closer to TD3, and adds automatic tuning of the entropy temperature, removing what was previously a sensitive, task-specific hyperparameter.

Across both lines of work, the common thread is that the Q-function is treated instrumentally: its accuracy is inferred indirectly, through the stability of the resulting policy, and the proposed fixes are evaluated by whether they improve final return rather than by how closely the underlying value estimates track the returns actually being realized. Even the deadly triad study, which analyzes Q-value estimates directly, does so within a single algorithm (DQN) rather than across algorithms with different stabilization mechanisms. This leaves open whether the error and instability patterns identified in that line of work are specific to value-based methods or also present, in the same or a different form, in the twin-critic actor-critic methods discussed above. The present project addresses this gap directly, by comparing predicted Q-values against Monte Carlo return estimates across DQN, SAC and TD3 on a shared pair of benchmark environments.

3. Methodology

3.1. Training Setup and Hyperparameters

We train three model-free RL algorithms from Stable-Baselines3 – **DQN** (discrete), **SAC** and **TD3** (continuous) – on two classic Gymnasium environments: **CartPole-v1** (natively discrete action space) and **Pendulum-v1** (continuous action space). Since DQN requires a discrete action space by definition, Pendulum is discretized for DQN via a custom `DiscretizeActionWrapper` into $N = 11$ evenly-spaced torque bins (via `np.linspace` over the bounds of the original Box action space); SAC and TD3, in contrast, operate directly on Pendulum’s native continuous action space. Each model is trained with the `MlpPolicy` and the default hyperparameters of the respective SB3 implementation.

Training and evaluation environments are initialized with different seeds (`seed` and `seed + 1000`, respectively) to avoid an accidental correlation between the training and evaluation trajectories. During training, an `EvalCallback` logs mean performance over 5 episodes every 2,000 steps and separately saves the best-performing checkpoint (`best_model.zip`) and the final model (`final_model.zip`); this is what the peak-vs-final degradation metric and the best-vs-final comparison in Section 3.3 are computed from. In addition, a checkpoint is saved every 10,000 to 50,000 steps depending on the run, which we later use to track the time course of Q-value bias over the course of training (checkpoint-progression analysis, Section 3.3).

3.2. Ground-truth Q-value Estimation via Monte Carlo Rollouts

To assess whether the learned Q-function systematically over- or underestimates the actually achievable return, we compare the network’s predictions against a Monte Carlo estimate of the true return.

3. Methodology

Q-value extraction. The function `get_q_value` is implemented algorithm-agnostically: for DQN, the Q-value is read directly from a forward pass of the `q_net` for the chosen action. For SAC/TD3 (twin-critic architecture), the state-action pair is passed through both critic networks; we report Q_1 and Q_2 individually as well as $\min(Q_1, Q_2)$ – the latter being the value SB3 also uses internally for the training target – and the mean $\bar{Q} = \frac{1}{2}(Q_1 + Q_2)$.

Monte Carlo return. As ground truth we use the discounted return of one complete, deterministically executed rollout episode starting from a state s_0 :

$$G(s_0) = \sum_{t=0}^T \gamma^t r_t, \quad (3.1)$$

using the discount factor γ of the respective trained model. We distinguish two estimation procedures:

- **estimate_return_at_state:** N repeated rollouts from the *same* state (via a `reset_fn` callback that puts the environment into a specific state) yield a low-variance, precise estimate for exactly one point in state space – intended for later grid/heatmap analyses of the Q-function.
- **estimate_bias_over_random_states:** one rollout per *random* start state, across $n = 50$ different starting states. This is the direct, algorithm-agnostic successor to the original `estimate_q_bias_dqn` approach, and considerably cheaper since it requires only one rollout per state instead of N .

For the bias analysis, immediately after reset we extract the model’s predicted Q-value for the first state-action transition at each random start state, and compare it to the discounted return actually achieved over the remainder of that episode.

3.3. Error Metric

As our primary error metric we use the **bias**

$$\text{bias}(s_0) = Q_{\text{pred}}(s_0) - G_{\text{MC}}(s_0) \quad (3.2)$$

per start state (positive = overestimation, negative = underestimation), averaged over $n = 50$ random states with a fixed seed (for comparability across runs). We report the mean and standard deviation of the bias per model/run.

3. Methodology

Because the absolute return scale varies substantially across environments and training stages (a barely-trained 10^4 -step model achieves a very different return range than a fully trained 10^6 -step model), we additionally normalize to the **relative bias**:

$$\text{relative_bias} = \frac{\text{bias}_{\text{mean}}}{|G_{\text{MC,mean}}|}, \quad (3.3)$$

which makes bias values comparable across budgets and environments.

We additionally draw on two further signals to disentangle genuine Q-value bias from mere training/evaluation noise:

1. **Eval-curve statistics** from the `evaluations.npz` data logged during training: the gap between peak and final performance (`degradation`, `degradation_pct`) serves as an instability/collapse indicator.
2. **Independent re-evaluation** of `best_model` vs. `final_model` with substantially more episodes ($n = 50$ instead of $n = 5$) and a seed distinct from the training-time evaluation schedule, to rule out that an apparent peak-vs-final degradation is merely correlated sampling noise. As a rough signal-to-noise measure (not a formal significance test), we report the gap between the two means in units of the combined standard error of the mean (SEM):

$$\text{SEM-ratio} = \frac{\bar{G}_{\text{best}} - \bar{G}_{\text{final}}}{\sqrt{\text{SEM}_{\text{best}}^2 + \text{SEM}_{\text{final}}^2}}. \quad (3.4)$$

For the 10^6 -step runs across all five seeds (one set per algorithm), bias is additionally computed across *every* saved checkpoint, giving a fine-grained view of how bias and performance evolve over the course of training.

4. Experiments and Results

4.1. Convergence and Bias at End of Training

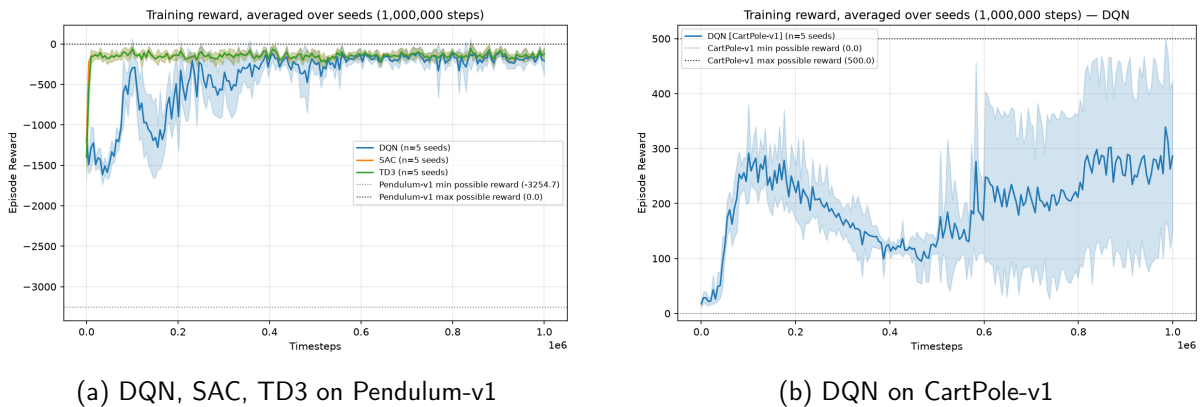


Figure 4.1.: Training reward curves, averaged over 5 seeds (shaded: ± 1 std), together with the theoretical minimum and maximum achievable episode reward on each environment (Pendulum-v1: $-3254.7/0.0$; CartPole-v1: $0/500$).

On Pendulum (Figure 4.1a), SAC and TD3 converge almost identically fast (within 50K steps) to a stable reward between -100 and -200 , with very low variance across seeds. DQN requires substantially more steps to converge (~ 400 K) and shows a considerably noisier learning curve with much larger inter-seed variance – plausible, since DQN here only operates over a discretized action set (`DiscretizeActionWrapper`, 11 bins), which limits control precision compared to the natively continuous SAC/TD3 policies. On CartPole (Figure 4.1b), DQN’s instability is even more pronounced and non-monotonic: after an initial rapid rise (0–100K steps, reaching ~ 250 – 290), mean performance drops back down to ~ 100 – 120 by ~ 450 K steps before recovering, and even at the end of training (1M steps) the mean reward plateaus around ~ 270 – 300 , well below the theoretical maximum of 500, with substantially growing inter-seed variance from ~ 500 K steps onward. This instability is consistent with the collapsed, poorly calibrated Q-function observed for the same setting in Figure 4.2: a Q-function that fails to distinguish between good and bad returns is a plausible symptom of the same underlying non-monotonic training dynamics, likely related to DQN’s known "deadly triad" of function approximation, bootstrapping, and off-policy learning.

4. Experiments and Results

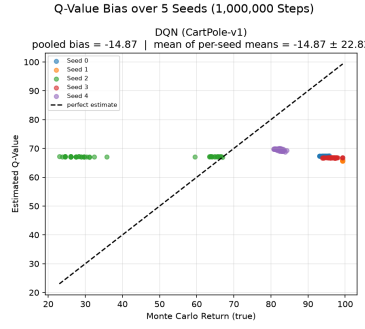


Figure 4.2.: Estimated Q-value vs. Monte Carlo return for DQN on CartPole-v1, pooled over 5 seeds ($n = 50$ states per seed).

Notably, the estimated Q-values remain almost constant at ~ 65 – 70 across the entire return range (20–100), instead of following the diagonal. The network thus barely distinguishes between "good" and "bad" episode starts – a considerably more severe and systematic issue than simple over- or underestimation (bias = -14.87 ± 22.82). This suggests a collapse of the learned Q-function, potentially driven by CartPole’s short, sparse reward structure (+1 per step) in combination with the training settings used here, and stands in contrast to the markedly better-calibrated DQN estimates on Pendulum (cf. Figure 4.3).

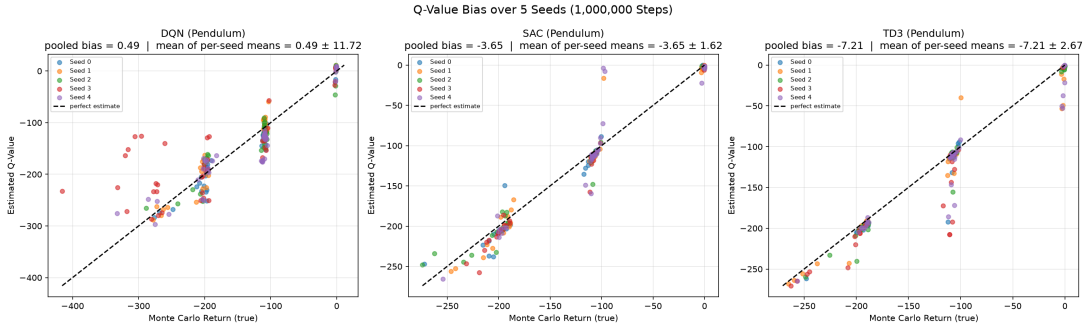


Figure 4.3.: Estimated Q-value vs. Monte Carlo return for DQN, SAC, and TD3 on Pendulum-v1, pooled over 5 seeds.

All three algorithms lie close to the perfect diagonal on average (bias: DQN 0.49, SAC -3.65 , TD3 -7.21). SAC and TD3 show the slight underestimation tendency expected from twin-critic methods ($\min(Q_1, Q_2)$), with low variance (± 1.62 and ± 2.67 , respectively). DQN is the least biased on average but has substantially higher variance (± 11.72) and a number of outliers at strongly negative returns. This is consistent with the greater instability of DQN on this environment observed in Figure 4.1a.

4. Experiments and Results

The figures above characterize bias only at the end of training, using non-deterministic, training-time reward logs (Figure 4.1) and final-checkpoint snapshots (Figures 4.2, 4.3). The following section instead tracks these same runs across every saved checkpoint, using deterministic re-evaluation aligned with the bias measurements, to examine how bias and policy performance co-evolve over the full course of training.

4.2. Policy Performance

To characterize how Q-value accuracy and policy performance evolve over the course of training, we combine three different views of the same runs (see 3.1), averaged across the five seeds per algorithm/environment: checkpoint progression and bias, a case study of a single anomalous checkpoint and training time evaluation of the 5M-step run (of DQN on CartPole). All bias values use the Monte Carlo procedure and normalization described in Section 3.2 and 3.3.

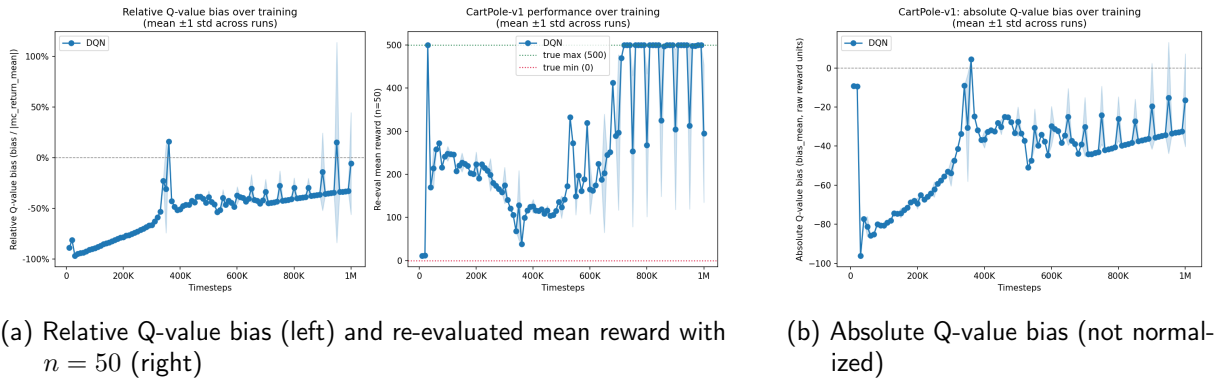


Figure 4.4.: Q-value bias over training on CartPole.

The relative bias starts strongly negative while the policy is still untrained, and then improves as the reward rises. But even once reward is consistently near-max, the bias plateaus at -30% to -50% rather than approaching 0. This means that apparent convergence of performance does not imply an accurate Q-function. The positive spike around 350K timesteps is a compound artifact. Only one of the 1M-step runs has a checkpoint at that step, and its own small raw deviation is tripled in relative terms by dividing over the low return at that training trough.

4. Experiments and Results

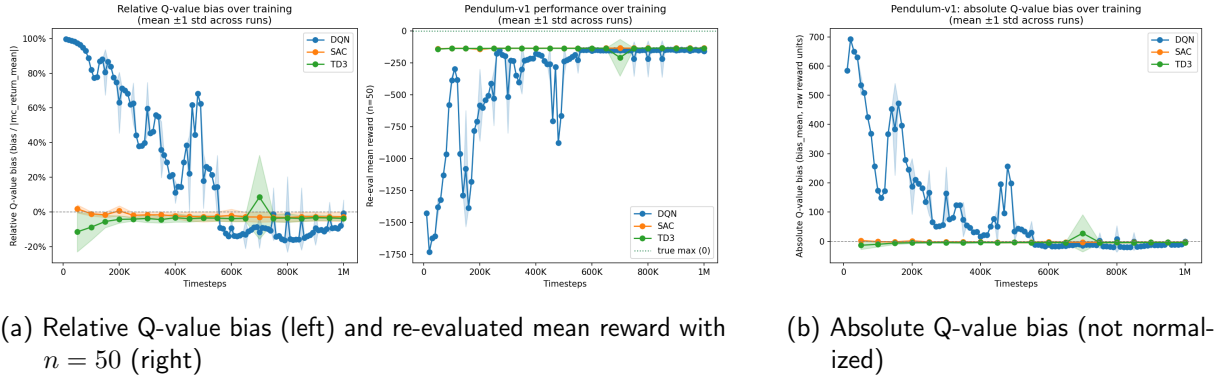


Figure 4.5.: Q-value bias over training on Pendulum.

SAC and TD3 reach small, stable bias and performance within the first 50K steps. DQN needs roughly 500K steps to reach comparable bias and only then matches their performance level. TD3’s spike at the 700Kth step is a checkpoint saved right after one unlucky episode, not a real policy collapse.

DQN starts overestimating on Pendulum, but underestimating on CartPole. The difference comes from the sign of each environment’s return. CartPole’s reward is always ≥ 0 and Pendulum’s is always ≤ 0 , so the same near-zero value looks highly optimistic against an untrained policy’s large negative return.

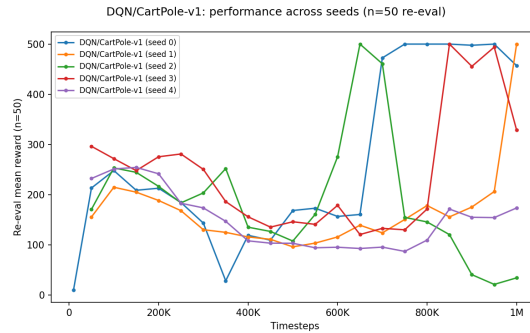


Figure 4.6.: Re-evaluated mean reward with $n = 50$ over training across different seeds

Four of five seeds spend most of training fluctuating at 100 to 300 before jumping to around 500 only between 600K and 1M; seed 4 never converges. Seed 2 reaches 500 at 650K timesteps but collapses to 30 by the final checkpoint.

4. Experiments and Results

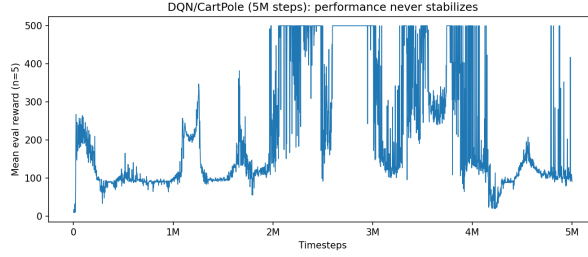


Figure 4.7.: Training-time evaluation reward with $n = 5$ over a 5M-step run

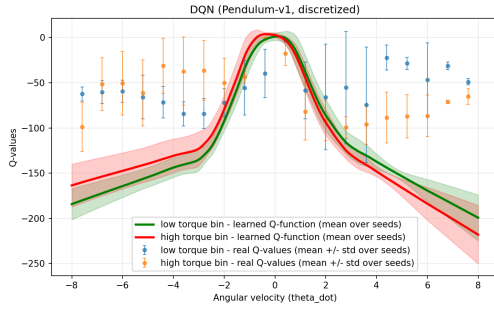
Reward repeatedly reaches the maximum of 500 for extended stretches and loses it again. The curve shows the same non-monotonic instability seen at the 1M-step run, persisting even at the 5M-step run.

4.3. Q-value against State across Environments

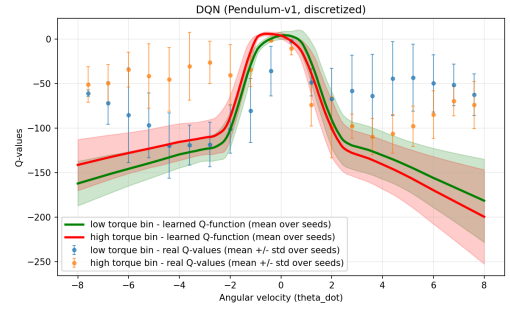
The Folder "Q-value_VS_State" is for visualizing how a model's learned Q-values compare to real rollout returns as a function of a specific state variable, rather than reducing bias to a single scalar per state as in the main Qvalue_bias.py.

For each of the DQN/CartPole, DQN/Pendulum, SAC/Pendulum and TD3/Pendulum models, real rollout data is collected by deterministic episodes (the default is 20) per seed, with the chosen state, action and return-to-go recorded per timestep. A 1D-state variable is extracted per trained model to plot against: cart position for CartPole and angular velocity for Pendulum. To make the visualisation of actions for CartPole and Pendulum more reader-friendly, actions are grouped into two categories: "push left/right" for CartPole and "low/high torque bin" for the discretised DQN on Pendulum, with continuous torque for SAC/TD3. Then the learned Q-function is evaluated on a smooth grid, with all the other state dimensions fixed (For the CartPole: set to 0; For the Pendulum: set to 1 for $\cos(\theta)$ and 0 for $\sin(\theta)$). Therefore, in the plot, the real Q-values are shown as error bars (mean $\pm$ standard deviation over seeds), and the learned Q-function is shown as a line with a shaded band showing the standard deviation (see 4.8a, 4.8b and 4.8c).

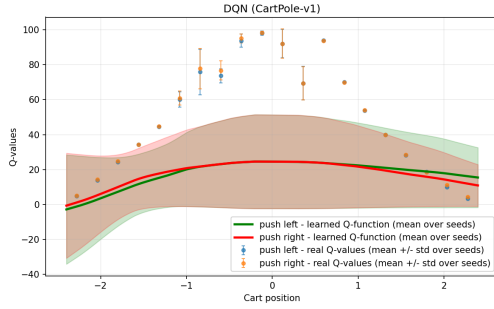
4. Experiments and Results



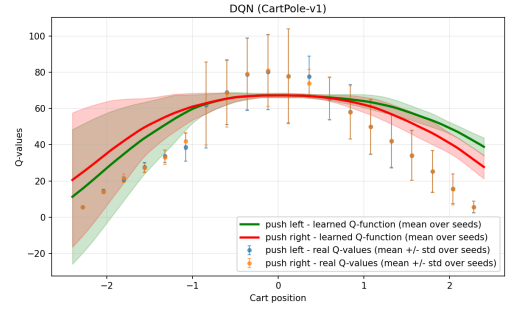
(a) Pendulum, best model



(b) Pendulum, final model



(c) CartPole, best model

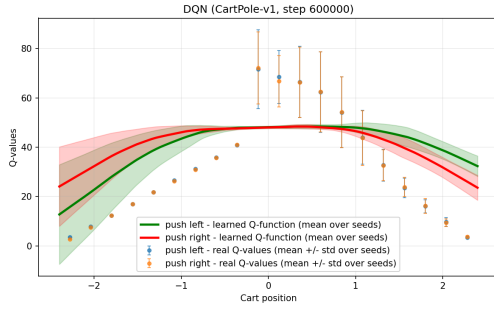


(d) CartPole, final model

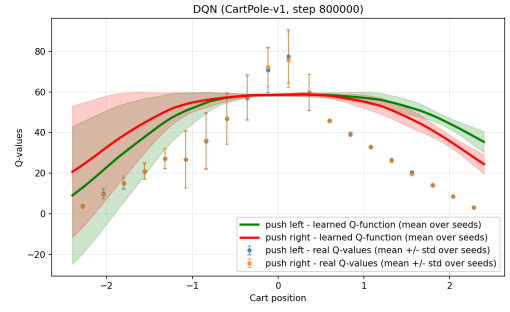
Figure 4.8.: Q-value against a physically meaningful state variable (angular velocity for Pendulum, cart position for CartPole) for DQN over 1M steps, comparing best and final model checkpoints.

Importantly, the best and final models of DQN on CartPole over 1M steps (see 4.8c and 4.8d) show a significantly different distribution of learned and real Q-values for cart positions. Therefore, it is worthwhile to examine the process of going through the checkpoint in order to track changes in Q-values. In 4.9a and 4.9b, learned curve peaks has risen from 48 to 59, while the underestimation gap of 20-40 at checkpoint 600,000 has shrunk to a gap of 15-17 at checkpoint 800,000, which indicates that the later learned curve tracks the real dots more closely over a wider central range (roughly $x \in [-0.6, 0.8]$), not just at the very peak. The real Q-value tends to have more diversity over the range $x \in [-0.2, 1.3]$ at checkpoint 600,000. But later at checkpoint 800,000, the real Q-value tends to have more diversity over the range $x \in [-1.1, 0.4]$.

4. Experiments and Results



(a) Checkpoint at 600,000



(b) Checkpoint at 800,000

Figure 4.9.: Q-value against cart position of DQN on CartPole over 1M steps, at intermediate checkpoints.

The observations and comparisons at checkpoints 600,000 and 800,000 match a known DQN-on-CartPole instability pattern that has already been documented in 4.6, which indicates that the DQN/CartPole run exhibits oscillations across different seeds.

4.4. Evolution of Q-function

The previous analyses show that DQN on CartPole exhibits persistent fluctuations in policy performance and that learned Q-values can differ substantially from empirical returns. To examine how the Q-function evolves during training, we evaluate five fixed states at every available checkpoint. For CartPole, the cart position and velocity are fixed to zero, while the pole angle and angular velocity vary across the five states shown in Figure 4.10. Both actions are evaluated.

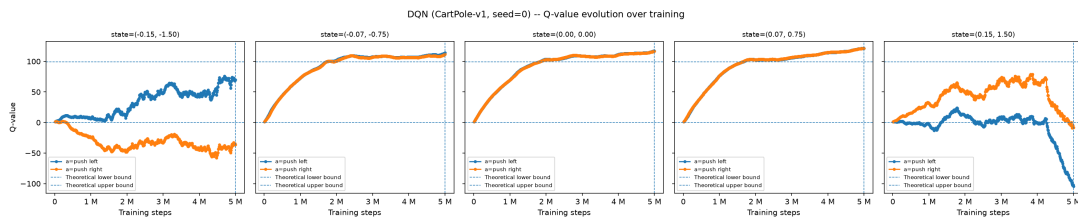


Figure 4.10.: Evolution of learned Q-values for five fixed CartPole states during 5M steps of DQN training. Cart position and velocity are fixed to zero; pole angle and angular velocity vary across the five states shown in the panels. Both discrete actions are evaluated.

For DQN on CartPole, the temporal evolution differs substantially between central and extreme states. The Q-values of the three central states increase rapidly during early training and become relatively stable after approximately 2M steps, whereas the two extreme states continue to exhibit larger fluctuations. This indicates that Q-function

4. Experiments and Results

stability is state-dependent. A possible explanation is that improving policies increasingly visit states near the upright configuration, resulting in greater representation of these states in the replay buffer, while extreme states receive less training signal.

The extreme states also show asymmetric action behavior. At state 1, the value of pushing left rises to roughly 70 while pushing right drifts toward -50 . At state 5, the pattern is largely mirrored, although the value of pushing right drops sharply after approximately 4M steps, from above 60 to near 0. Consequently, the relative ranking of actions can change even after substantial training. Moreover, several learned Q-values exceed the theoretically possible discounted-return range. Since CartPole provides a reward of 1 per step and episodes last at most 500 steps,

$$0 \leq G \leq \sum_{t=0}^{499} \gamma^t = \frac{1 - \gamma^{500}}{1 - \gamma} \approx 99.34,$$

for $\gamma = 0.99$. Other learned values become negative, showing that the learned Q-function can deviate substantially from valid discounted returns.

The corresponding Pendulum trajectories are provided in Appendix A. DQN again shows state-dependent temporal variation, with the central state becoming stable earlier than the off-center states. In contrast, SAC and TD3 show substantially smaller temporal variation: SAC is already close to its final values at the first checkpoint, while TD3 reaches a stable range after approximately 200K–400K steps. Across the selected Pendulum states, the evaluated action values are also relatively close to one another.

The temporal bias analysis in Figures 4.5b and 4.4b is consistent with these observations. Pendulum exhibits relatively small and stable absolute bias for all three algorithms after roughly 600K steps, whereas DQN on CartPole retains substantial oscillations. Overall, these results show that policy performance can stabilize while the learned Q-function continues to change, particularly in DQN on CartPole.

5. Discussion

5.1. Deviation from Theoretical Optimality

On Pendulum, all three algorithms end up close to the true Monte Carlo return on average (Figure 4.3) and absolute bias for all three converges toward zero within roughly 600K steps (Figure 4.5b). Optimality is therefore approached closely whenever training is allowed to run long enough.

DQN on CartPole behaves differently. Pooled over all five seeds, its Q-values cluster almost constantly around 65 to 70 regardless of the actual realized return, which ranges from 20 to 100 (Figure 4.2). This is a qualitatively different failure from a simple offset, since the network barely tracks how good or bad an episode start actually was. Over the course of a single training run (Figure 4.4a), the relative bias never approaches zero even once the policy performs near-optimally from step 750K onward. It instead plateaus at -30% to -50% for the remainder of the training. Apparent convergence of policy performance therefore does not imply convergence of the underlying Q-function toward an accurate value estimate.

The discounted return for CartPole is bounded by $0 \leq Q(s, a) \leq 99.34$ for $\gamma = 0.99$. Figure 4.10 shows the learned Q-function violating both ends of this range: central states rise above the theoretical maximum (overestimation) while extreme states fall below zero (underestimation). The learned function is thus not merely inaccurate in places, but at times represents values that are impossible for the environment.

5.2. Error Localization in State Space

Tracking five fixed states across training (see 4.4) shows the same pattern on both environments. Central states (small pole angle/angular velocity for CartPole, state (0, 0) for Pendulum) stabilize quickly and fluctuate only slightly afterward, while the more extreme states show substantially larger and more persistent fluctuations throughout training (Figures 4.10, A.1) — the same central-overestimation/extreme-underestimation split noted in Section 6.1. A plausible mechanism is that the replay buffer’s state distribution

5. Discussion

shifts as the policy improves: successful trajectories spend most of their time near central states, so extreme states are sampled less often and receive comparatively stale training signal.

The extreme states of CartPole additionally show an asymmetry between actions. At state 5, the "push right" Q-value collapses from above 60 to near 0 after roughly 4M steps while "push left" does not, indicating that the network keeps reorganizing which action looks better specifically at the states it sees least often.

This central-vs-extreme gradient appears specific to DQN. On Pendulum, SAC's Q-values are already flat and stable across all five states from the first checkpoint, and TD3's early transient is not reported as being concentrated at particular states (Figures A.2, A.3). When an action has little true effect on the return (Pendulum's small, symmetric torque range), the three action curves within a state are nearly indistinguishable for all three algorithms, so weak error is not confined to DQN when the underlying quantity being estimated is itself nearly action-independent.

A complementary view comes from plotting the learned Q-function directly against a single, physically meaningful state variable (cart position for CartPole, angular velocity for Pendulum) rather than reducing it to a handful of fixed states (Section 4.3). For DQN on CartPole, this view shows the best and final 1M-step models producing markedly different Q-value distributions across cart position (Figures 4.8c, 4.8d). Tracking the same curve across checkpoints 600K and 800K (Figures 4.9a, 4.9b) shows the mismatch easing within that window: the learned peak rises from 48 to 59 and the underestimation gap over the central range narrows from 20-40 to 15-17, but the region where real rollout returns vary most also shifts, from roughly $x \in [-0.2, 1.3]$ to $x \in [-1.1, 0.4]$. So it is not only the size of the error that changes between checkpoints, but which part of the state space is least predictable in the first place. This matches the broader, continuing oscillation documented for this run (Figure 4.7).

5.3. Q-Functions After Extended Training

For SAC and TD3 on Pendulum, once past their respective initial transients, the learned Q-function does closely track the real Monte Carlo return, with a small and stable negative bias consistent with the twin-critic $\min(Q_1, Q_2)$ target both algorithms use (Figure 4.3).

DQN on CartPole illustrates the opposite case. Its bias never shrinks toward zero even once the policy is performing near-optimally (Figure 4.4a), and in its most severe form the predicted Q-values barely change with the state at all, clustering around the same value regardless of the return actually obtained (Figure 4.2). Even after extended training

it retains a persistent -30% to -50% relative bias (4.4b) rather than approaching zero. So a trained Q-function does not automatically become an accurate, state-specific value estimate just because training has continued and the policy has converged — here it instead settles into something closer to a rough, average expected return for policies like this one than an estimate of the value of any particular state. Nor does long training imply convergence: the extreme-state action reversals noted above emerge as late as 4M steps into a 5M-step run, showing that some parts of the Q-function are still actively changing long after the policy’s own performance has plateaued.

5.4. Recurring Error and Instability Patterns

Some patterns recur regardless of algorithm and are best explained by the environment itself. The sign of the earliest Q-value bias, for example, is set entirely by the sign of the environment’s return rather than by any algorithmic property: the same near-zero-initialized network overestimates on Pendulum, whose return is bounded below zero, and underestimates on CartPole, whose return is bounded above zero (see 4.2). SAC and TD3 also share a small, stable negative bias on Pendulum (Figure 4.3), consistent with the $\min(Q_1, Q_2)$ target both algorithms were built around.

Most of the more severe instability, however, is specific to DQN. On Pendulum, DQN needs roughly five to ten times as many steps as SAC or TD3 to converge, shows a visibly noisier learning curve, larger inter-seed variance, and occasional outliers at strongly negative returns (Figure 4.1a) — plausibly a consequence of operating over the 11-bin discretized action set rather than natively continuous control. On CartPole, DQN is unstable at every time scale examined: within a single run (the checkpoint progression oscillations in Figure 4.4a), across seeds of the same 1M-step run (only seed 1 remains stably at the maximum, seed 2 collapses at 650K, seed 4 never converges in Figure 4.6), and even at five times the training budget (the 5M-step run repeatedly regains and loses the maximum return, ending at 80 to 120 despite reaching 500 in Figure 4.7). The state-dependent stability gradient discussed above is likewise documented for DQN on both environments but not reported for SAC or TD3, reinforcing that it reflects DQN’s value-based, discrete-action structure rather than a universal property of learned Q-functions.

Overall, SAC and TD3 differ from DQN in degree rather than in kind on Pendulum. Both eventually reach comparable calibration and stability, but no comparable stabilization is observed for DQN on CartPole within any of the training budgets tested here.

6. Conclusion and Outlook

6.1. Conclusion

This project investigated whether Q-functions learned by model-free reinforcement learning algorithms accurately reflect the returns achieved by their policies. The results show that policy performance and Q-function accuracy can differ substantially and should therefore be evaluated separately.

On Pendulum, DQN, SAC, and TD3 ultimately learn relatively well-calibrated Q-functions, with predicted values closely matching Monte Carlo returns. In contrast, DQN on CartPole exhibits persistent value-estimation errors even after achieving near-optimal policy performance. Predicted Q-values remain weakly related to realized returns, and some estimates even fall outside the theoretically valid return range.

The experiments also demonstrate that Q-function errors vary across the state space. Frequently visited states tend to stabilize earlier, while more extreme states can exhibit substantial fluctuations and changing action preferences long after strong policy performance has been reached. Extending training alone does not necessarily eliminate this instability.

Overall, these findings demonstrate that high policy reward does not imply an accurate or converged Q-function. Directly comparing learned Q-values with empirical returns therefore provides an important complementary diagnostic for assessing the reliability and stability of reinforcement learning algorithms.

6.2. Outlook

Future work should extend the analysis to a broader range of environments and algorithms to enable more direct comparisons under identical conditions. A more extensive Monte Carlo evaluation across a denser set of states could also provide a more precise characterisation of Q-function errors and their relationship to state visitation and replay-buffer coverage.

6. Conclusion and Outlook

Another direction for future research is to investigate whether the observed Q-function instability persists in higher-dimensional and more complex reinforcement learning tasks. Investigating the relationship between value-function accuracy, policy changes, replay-buffer composition and training dynamics could help to explain why strong policy performance can coexist with inaccurate value estimates. Ultimately, this could lead to the development of evaluation and training methods that consider not only the achieved reward, but also the reliability of the learned value function.

7. Bibliography

- Fujimoto, Scott, Herke van Hoof, and David Meger (2018). “Addressing Function Approximation Error in Actor-Critic Methods”. In: *Proceedings of the 35th International Conference on Machine Learning*, pp. 1587–1596.
- Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine (2018). “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”. In: *Proceedings of the 35th International Conference on Machine Learning*, pp. 1861–1870.
- Haarnoja, Tuomas, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine (2018). “Soft Actor-Critic Algorithms and Applications”. In: *arXiv preprint arXiv:1812.05905*. URL: <https://arxiv.org/abs/1812.05905>.
- Hasselt, Hado van (2010). “Double Q-learning”. In: *Advances in Neural Information Processing Systems 23*. URL: https://proceedings.neurips.cc/paper_files/paper/2010/file/091d584fced301b442654dd8c23b3fc9-Paper.pdf.
- Hasselt, Hado van, Yotam Doron, Florian Strub, Matteo Hessel, Nicolas Sonnerat, and Joseph Modayil (2018). “Deep Reinforcement Learning and the Deadly Triad”. In: *arXiv preprint arXiv:1812.02648*. URL: <https://arxiv.org/abs/1812.02648>.
- Hasselt, Hado van, Arthur Guez, and David Silver (2016). “Deep reinforcement learning with double Q-Learning”. In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*. AAAI’16. Phoenix, Arizona: AAAI Press, pp. 2094–2100.
- Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis (2015). “Human-level control through deep reinforcement learning”. In: *Nature* 518, pp. 529–533. DOI: 10.1038/nature14236.

Appendix A. Additional Figures

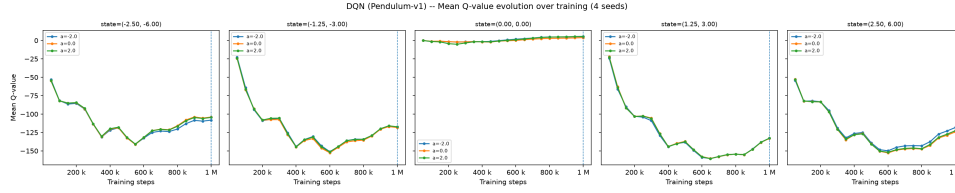


Figure A.1.: Evolution of learned Q-values for five fixed Pendulum states during 1M steps of DQN training. States are represented by $(\theta, \dot{\theta})$; the corresponding three-dimensional network observation is $[\cos \theta, \sin \theta, \dot{\theta}]$.

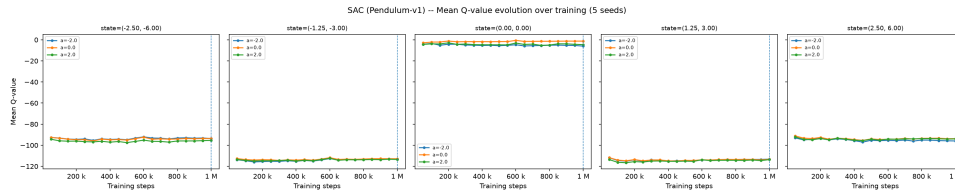


Figure A.2.: Evolution of learned Q-values for five fixed Pendulum states during 1M steps of SAC training. States are represented by $(\theta, \dot{\theta})$; the corresponding network observation is $[\cos \theta, \sin \theta, \dot{\theta}]$.

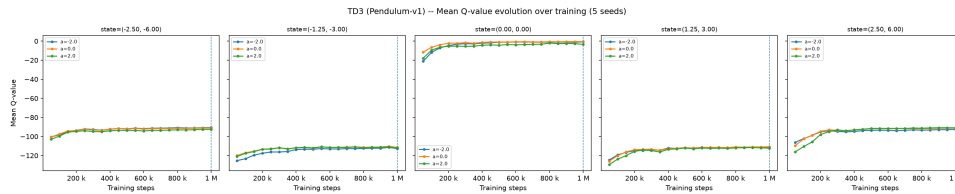


Figure A.3.: Evolution of learned Q-values for five fixed Pendulum states during 1M steps of TD3 training. States are represented by $(\theta, \dot{\theta})$; the corresponding network observation is $[\cos \theta, \sin \theta, \dot{\theta}]$.

A. Additional Figures

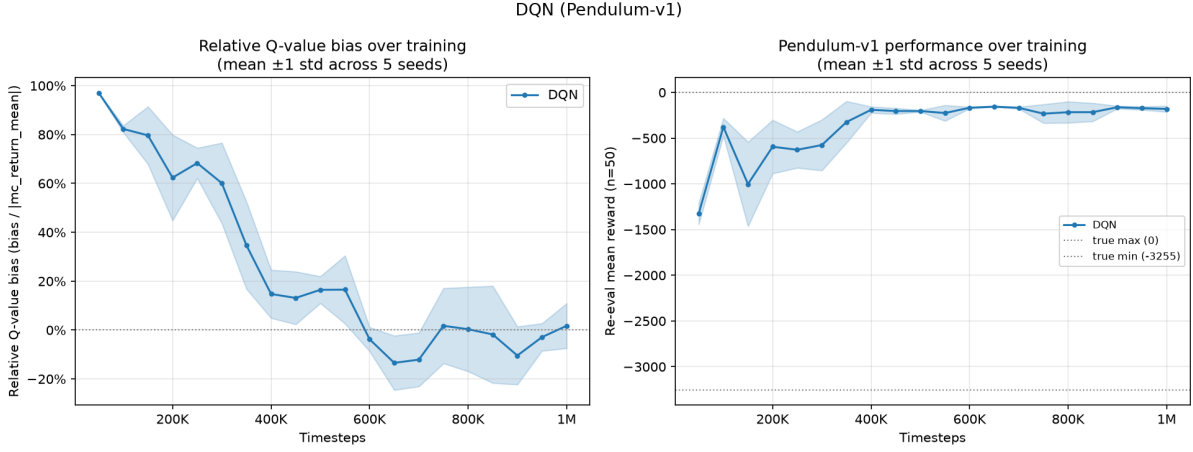


Figure A.4.: Relative Q-value bias (left) and re-evaluated mean reward with $n = 50$ (right) over training for DQN on Pendulum-v1, mean ± 1 std across 5 seeds.

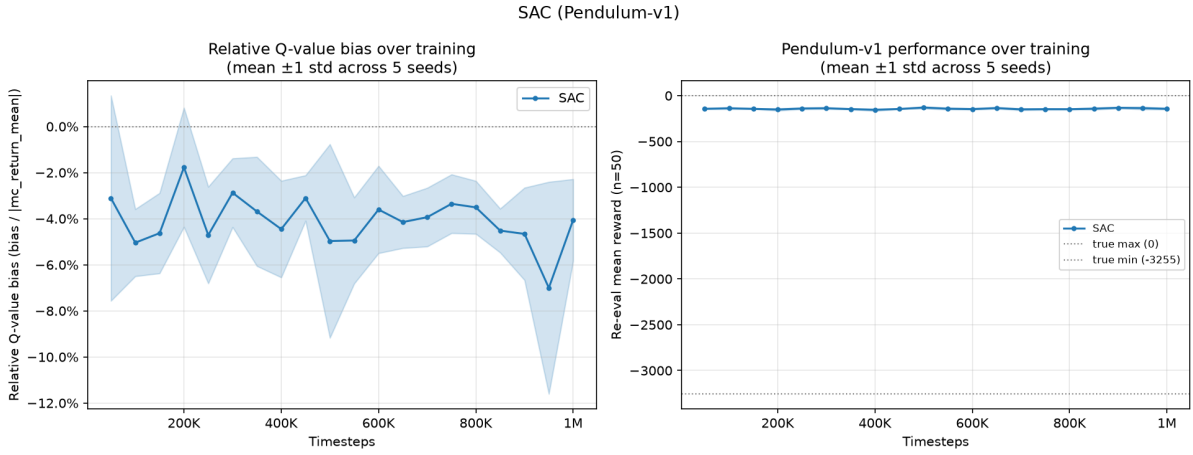


Figure A.5.: Relative Q-value bias (left) and re-evaluated mean reward with $n = 50$ (right) over training for SAC on Pendulum-v1, mean ± 1 std across 5 seeds.

5

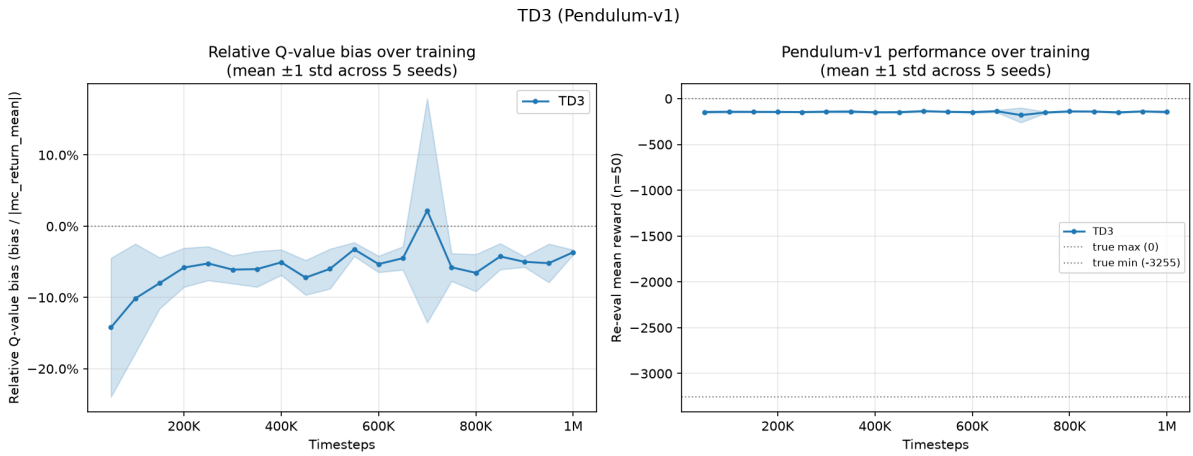


Figure A.6.: Relative Q-value bias (left) and re-evaluated mean reward with $n = 50$ (right) over training for TD3 on Pendulum-v1, mean ± 1 std across 5 seeds.

Appendix B. Selected Algorithms

This work investigates three model-free, off-policy reinforcement learning algorithms: Deep Q-Network (DQN), Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3). They were selected because they represent different approaches to learning and using Q-functions while sharing the use of function approximation and experience replay (Mnih et al., 2015; Haarnoja, Zhou, Abbeel, et al., 2018; Fujimoto et al., 2018).

Deep Q-Network (DQN) is a value-based algorithm for discrete action spaces. It represents the action-value function with a neural network $Q_\theta(s, a)$ and derives the policy by selecting the action with the highest predicted value,

$$\pi(s) = \arg \max_a Q_\theta(s, a). \quad (\text{B.1})$$

The network is trained using temporal-difference targets, while experience replay and a separate target network are used to improve training stability (Mnih et al., 2015). Since the network directly provides a Q-value for every available action, DQN allows individual action values to be inspected directly.

Soft Actor-Critic (SAC) is an actor-critic algorithm for continuous action spaces. It learns a stochastic policy together with two critic networks that estimate the Q-function. In addition to maximizing expected return, SAC maximizes policy entropy, encouraging exploration during training (Haarnoja, Zhou, Abbeel, et al., 2018). The minimum of the two critic estimates is used for the target, reducing overestimation caused by function approximation. SAC therefore provides a continuous-control setting in which Q-values are learned by critics rather than directly determining the action through a discrete maximization.

Twin Delayed Deep Deterministic Policy Gradient (TD3) is also an actor-critic algorithm for continuous action spaces. It uses a deterministic actor and two critics and was specifically proposed to address function approximation errors in actor-critic methods (Fujimoto et al., 2018). TD3 reduces overestimation by using the smaller of the two critic estimates, delays actor updates, and applies smoothing noise to target actions. These

B. Selected Algorithms

mechanisms make TD3 particularly relevant to the present study, since the accuracy of its learned Q-functions is directly related to one of the main motivations behind the algorithm.

The three algorithms consequently differ in their policy representation, action-space type, and treatment of value-estimation errors. DQN uses a value-based approach with discrete actions, while SAC and TD3 use actor-critic architectures for continuous actions. SAC employs a stochastic entropy-regularized policy, whereas TD3 uses a deterministic policy and additional mechanisms to stabilize critic learning. These differences allow the experiments to investigate whether Q-value errors exhibit algorithm-specific or more general patterns.

Appendix C. Benchmark Environments

Two low-dimensional control environments are used: **CartPole-v1** and **Pendulum-v1**. Their small observation spaces allow systematic sampling of states for Q-function analysis.

CartPole-v1 consists of a cart moving horizontally with a pole attached through a rotational joint. The objective is to keep the pole upright by moving the cart left and right. Formally the **observation** is

$$s = [x, \dot{x}, \theta, \dot{\theta}], \quad (\text{C.1})$$

representing cart position, cart velocity, pole angle and angular velocity. The agent chooses between **two discrete actions**, pushing the cart left or right. Each step yields a **reward** of 1 until the episode **terminates** when the cart or pole leaves its allowed range, or is truncated after 500 steps. This environment is used for DQN.

Pendulum-v1 is a continuous-control environment in which the agent applies torque to a pendulum attached to a fixed point. The **observation** is

$$s = [\cos(\theta), \sin(\theta), \dot{\theta}], \quad (\text{C.2})$$

with torque $\tau \in [-2, 2]$. The **reward** is

$$r = -(\theta^2 + 0.1\dot{\theta}^2 + 0.001\tau^2), \quad (\text{C.3})$$

and episodes are truncated after 200 steps without early termination. SAC and TD3 use the continuous action space, while DQN uses 11 evenly spaced discrete torque values.